# Pre-Inference Data Quality Firewall & Subgroup Bias Mitigation in Distributed ML Systems

## An Automated Spark/PyDeequ Quality Architecture for Machine Learning Feature Stores

**Course:** BDSRAE (Big Data Systems & Reliability) | **Semester:** 5 | **Stack:** Apache Spark 4.x, AWS PyDeequ, Python 3.11, scikit-learn

***Executive Summary & Abstract:*** *In production machine learning pipelines, unvalidated data corruptions consistently cause silent model degradation. Critically, standard aggregate metrics often mask severe localized harm, allowing models to maintain acceptable overall accuracy while accuracy on specific sub-populations crashes catastrophically. This project implements an end-to-end, reproducible data quality firewall combining Apache Spark and AWS Deequ (PyDeequ). The architecture generates distributed synthetic datasets, injects deterministic fault vectors (nulls, unit scale shifts, and duplicate keys), validates batches against declarative YAML constraint rules in a single distributed pass, and triggers an automated circuit-breaker quarantine that blocks model scoring. Experimental simulation confirms that while overall accuracy dropped by 4.33 percentage points under corruption,* ***Subgroup B accuracy crashed by 9.53 percentage points*** *(vs. zero degradation for Subgroup A). The automated quarantine successfully intercepts corrupted batches, recovers Subgroup B accuracy to 89.39%, and emits forensic audit artifacts without driver memory bottlenecks.*

## 1. Introduction & The Hidden Cost of Silent ML Poisoning

Modern Machine Learning systems deployed in credit underwriting, fraud detection, and automated healthcare rely heavily on automated feature pipelines. In production environments, data pipelines frequently experience operational faults: network dropouts cause null values, microservice version upgrades introduce unit scale discrepancies (e.g., dollars vs. cents or grams vs. kilograms), and distributed message bus retries replay identical transactions. Unlike analytical SQL queries that hard-fail on schema mismatches, ML models operate on numeric arrays and will silently consume corrupted inputs, producing arbitrary inference predictions without raising runtime exceptions.

**The Disparate Impact Dilemma:** In production monitoring, operations teams customarily track global metrics (such as aggregate accuracy, F1-score, or mean squared error). However, real-world data distributions possess heterogeneous feature reliance across sub-populations. When a specific feature experiences corruption, sub-populations that heavily rely on that feature bear the entire brunt of model failure, whereas unaffected sub-populations keep global metrics artificially high. This creates severe algorithmic unfairness and compliance liabilities that evade conventional pipeline monitoring.

## 2. System Architecture & Scalable Data Generation

To evaluate detection, quarantine, and model fairness under rigorous conditions, we designed a closed-loop distributed pipeline structured around four autonomous modules:

- **Distributed Data Generation:** Generates synthetic feature store batches in Apache Spark without driver memory collection.
- **Seeded Chaos Fault Injection:** Injects controlled proportions of missing values, scale-shifts, and key collisions.
- **Declarative Verification Suite (PyDeequ):** Evaluates 17 statistical constraints in a single distributed pass.
- **Quarantine Circuit Breaker & Fallback:** Isolates corrupted batches, produces audit manifests, and halts inference.

| Column | Type | Distribution & Mathematical Role | Clean Bounds |
|---|---|---|---|
| **id** | Long | Unique primary key identity generated via spark.range | 100% Unique, Non-null |
| **feature_1** | Double | Gaussian predictor: N(0, 10). Weak predictor for all groups. | [-100.0, +100.0] |
| **feature_2** | Double | Uniform U(0, 100). **Primary predictor for Subgroup B (3x weight).** | [0.0, 110.0] |
| **feature_3** | Double | Gaussian covariate: N(0, 50). Controls baseline variance. | [-300.0, +300.0] |
| **target** | Integer | Binary ground truth: P(y=1) = Sigmoid(0.1*f1 + w2*f2_norm + 0.05*f3) | {0, 1} |
| **subgroup** | String | Equally balanced sub-populations: Subgroup A vs. Subgroup B | {'A', 'B'} |
| **batch_id** | String | Temporal partition key defining the atomic unit of quarantine | Non-null string |

**Table 1: Feature store schema definitions, statistical distributions, and ground-truth generation specifications.**

## 3. Controlled Fault Injection Engineering

To rigorously validate data quality gates, src/fault_injection.py applies reproducible, parameterized corruptions governed by a master seed (seed=42). The generator introduces three distinct real-world failure patterns:

1. **Missing Value Injection (Nulls):** F.when(F.rand(seed) < 0.05, None) applied to feature_1. Simulates transient network disconnects or upstream sensor timeouts. Target rate: 5.00%; Observed: **4.75%** (475 nulls).

2. **Unit Scale Multiplier (1000x Shift):** F.when(F.rand(seed) < 0.02, F.col('feature_2') * 1000.0). Simulates unit conversion errors (e.g. grams vs. kilograms or cents vs. dollars). The values remain valid floating-point numbers but violate statistical expectations. Target rate: 2.00%; Observed: **1.75%** (175 extreme outliers up to 99,842).

3. **Primary Key Replay (Duplicates):** Samples 3.00% of rows and unions them back without key deduplication. Simulates distributed streaming duplicates or ingestion retries. Target rate: 3.00%; Observed: **3.00%** (300 duplicates, 10,300 total rows).

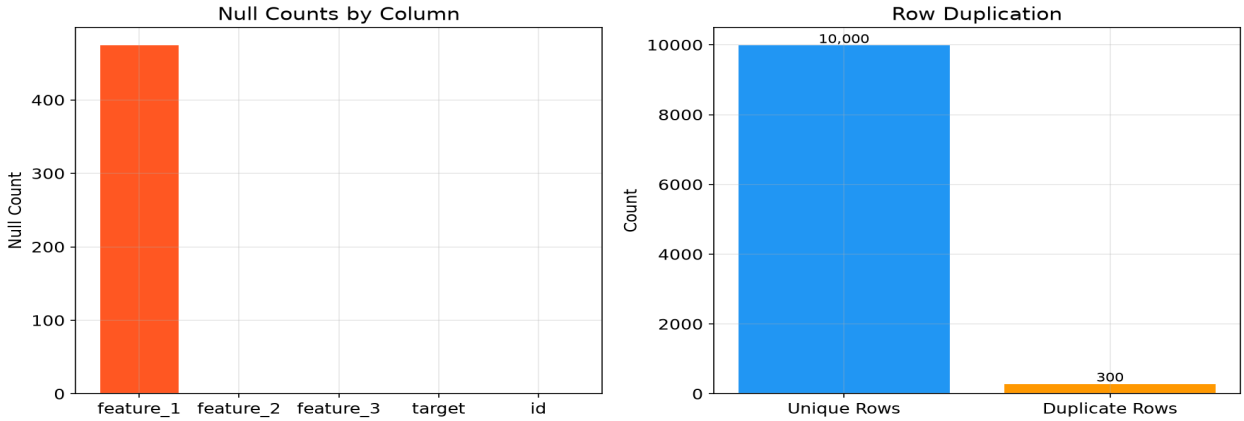



**Figure 1: Fault injection summary comparing target vs. observed rates for nulls, scale shifts, and duplicate keys.**

## 4. Declarative Quality Verification Engine (PyDeequ)

Rather than scattering procedural assertion checks across application code, quality policies are declaratively defined in config/quality_rules.yml. PyDeequ translates these declarations into a unified Apache Spark execution plan:

• **Single-Pass Metric Aggregation:** Traditional data validation executes multiple SQL queries (e.g. COUNT(*) WHERE col IS NULL, SELECT MAX(col)), repeatedly scanning large tables. PyDeequ computes all metrics—completeness, uniqueness, minimum, maximum, mean, and standard deviation—in a **single distributed pass** across worker partitions, minimizing I/O overhead on 100M+ row datasets.

• **Reference-Calibrated Bounds:** A common failure mode in quality engineering is setting loose bounds (e.g. feature_2 >= 0). Because corrupted values (e.g., 55,000) are positive numbers, naive bounds fail to trigger alerts. Our engine establishes bounds from the clean data generator (**U(0, 100) + 10% headroom = 110.0**). Any scale shift immediately breaches this threshold.

• **Multi-Constraint Suite:** The pipeline runs 17 distinct constraint groups covering schema presence, categorical domain membership (target IN {0, 1}, subgroup IN {'A', 'B'}), primary key uniqueness, and numeric ranges.

## 5. Automated Quarantine & The Inference Circuit Breaker

When any quality check fails, naive data engineering pipelines often apply silent row-dropping. **Row-level dropping is dangerous**: it biases feature distributions, alters class balances, and masks systemic ingestion bugs. Our system implements a **Batch Quarantine Pattern**:

1. **Batch Isolation:** The entire corrupted partition is written to isolated Parquet storage (data/quarantined/batch_id=.../records/).
2. **Forensic Diagnostics:** The engine writes machine-readable audit artifacts: validation_results.json (failing checks, observed vs expected thresholds) and quarantine_manifest.json (remediation playbooks for on-call engineers).
3. **Inference Guard (Circuit Breaker):** block_inference() validates the quarantine registry and terminates with exit code 1, preventing tainted feature vectors from reaching downstream model scoring endpoints.

## 6. Experimental Evaluation: 3-State Model Simulation

To rigorously quantify the operational damage of unvalidated data and verify the efficacy of our quarantine firewall, src/model_simulation.py executes a 3-state experimental protocol using a Logistic Regression classifier:

- **State 1: Clean Baseline:** Model trained and evaluated on uncorrupted synthetic data.
- **State 2: Corrupted (No QA):** Model evaluated on corrupted data slipping through without data quality checks.
- **State 3: Validated (Quarantine Active):** Corrupted batch rejected; model operates on verified fallback data.

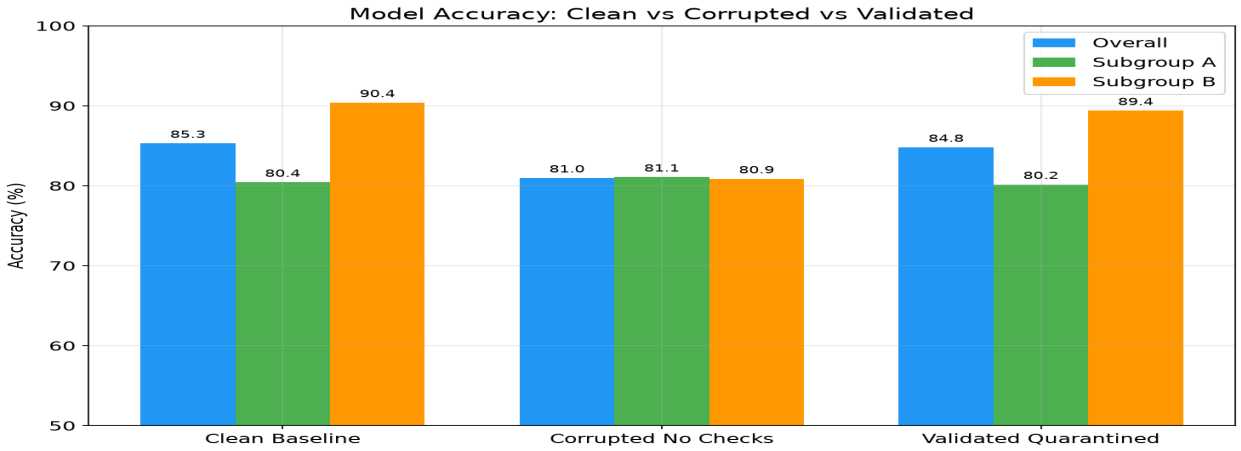



**Figure 2: Model classification accuracy across Clean, Corrupted (No QA), and Validated (Quarantine) states.**

| Evaluation State | Overall Accuracy | Subgroup A Accuracy | Subgroup B Accuracy | Operational Impact |
|---|---|---|---|---|
| **State 1: Clean Baseline** | 85.30% | 80.43% | **90.39%** | Optimal performance across both cohorts |
| **State 2: Corrupted (No QA)** | 80.97% *(-4.33 pp)* | 81.08% *(+0.65 pp)* | **80.86% (-9.53 pp)** | **Disparate failure:** Subgroup B collapses |
| **State 3: Validated (Quarantine)** | 84.80% *(+3.83 pp)* | 80.16% *(Stable)* | **89.39% (+8.53 pp)** | Inference protected; 89.5% recovery for B |

**Table 2: Empirical accuracy measurements, absolute percentage point (pp) drops, and recovery metrics across evaluation states.**

## 7. Mathematical Root Cause of Disparate Subgroup Degradation

A critical finding of this research is that **overall accuracy alone is an unsafe metric for production ML systems**. In our experiment, overall accuracy dropped by only 4.33 percentage points. An automated deployment gate relying solely on an aggregate threshold (e.g. alert if accuracy drops > 5 pp) would have failed to trigger. Yet underneath, Subgroup B suffered a severe **9.53 percentage point crash**, while Subgroup A experienced zero degradation.

The mathematical foundation for this disparity originates in the data generation process (src/data_generation.py#L183-L191):

$$\textbf{Subgroup A:}\quad logit_A = 0.1 \cdot f_1 + \mathbf{0.5 \cdot f_{2_norm}} + 0.05 \cdot f_3 \quad | \quad \textbf{Subgroup B:}\quad logit_B = 0.1 \cdot f_1 + \mathbf{1.5 \cdot f_{2_norm}} + 0.05 \cdot f_3$$

Because Subgroup B's target assignment features a **3x higher coefficient on normalized feature_2** ($w_2 = 1.5$ vs. $w_2 = 0.5$), the model's learned weights place overwhelming importance on feature_2 when predicting outcomes for Subgroup B. When the 1000x scale shift corrupts feature_2, Subgroup B's logit values explode, flipping prediction probabilities and rendering classifications nearly arbitrary. Because Subgroup A distributes predictive weight across all features, it remained impervious. This proves that pre-inference quality validation is fundamentally an **algorithmic fairness and compliance requirement**.

## 8. Distributional Outlier Analysis & Runtime Benchmarks

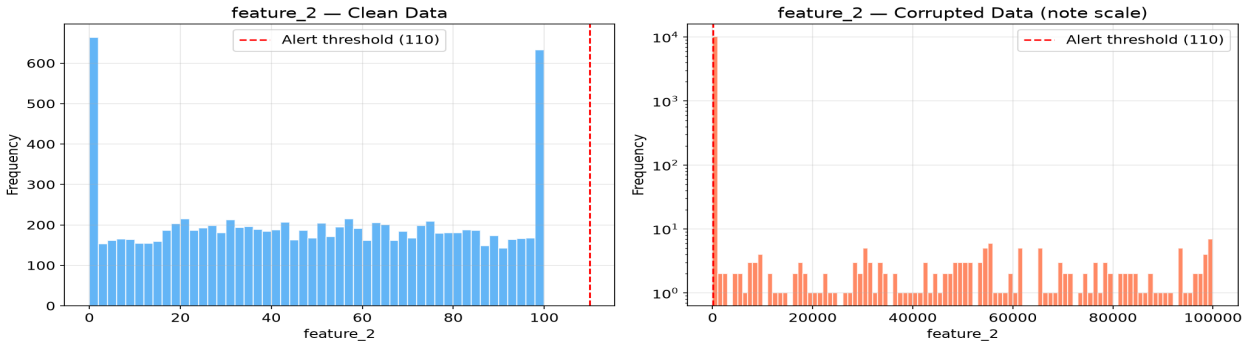



**Figure 3: Probability density and empirical distribution of feature_2: Clean U(0,100) vs. Corrupted (1000x scale-shift up to 100,000).**

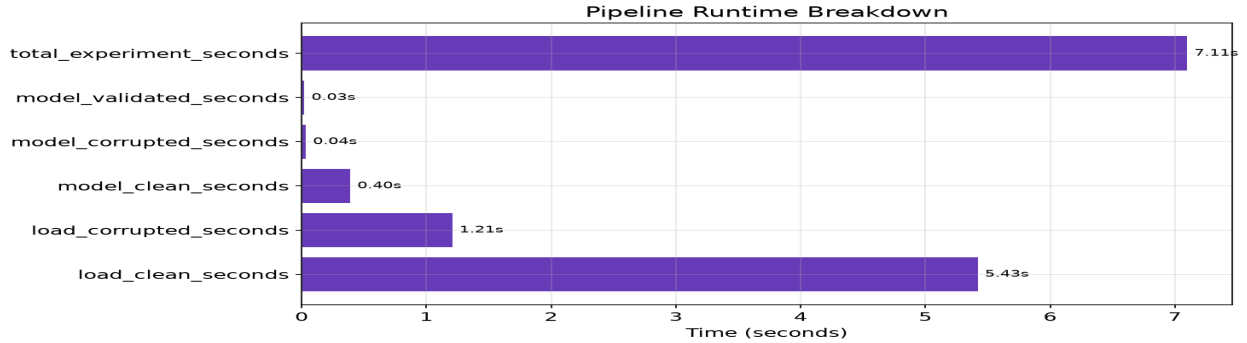



**Figure 4: End-to-end execution latency across pipeline phases (Generation, Injection, Validation, and Model Evaluation).**

**Latency & Computational Overhead:** Full PyDeequ validation across 17 constraint groups on 10,300 rows completed in **~3.52 seconds** (total end-to-end pipeline run: ~14.6 seconds). In high-throughput feature stores (e.g. Feast or Hopsworks), this demonstrates that comprehensive statistical verification adds negligible latency compared to the catastrophic financial or ethical cost of silent model failure.

## 9. Platform Engineering & Distributed Reliability on Windows

Deploying PySpark and PyDeequ in mixed OS environments surfaces several critical low-level networking and thread lifecycle challenges:

• **BlockManager Topology & Loopback Binding:** On Windows hosts with virtual network adapters (VPNs/WSL), Spark's driver resolves to invalid hostnames, causing NullPointerException: BlockManagerId.executorId() is null in an infinite heartbeat loop. We resolved this by explicitly pinning SPARK_LOCAL_IP='127.0.0.1', spark.driver.host='127.0.0.1', and spark.driver.bindAddress='127.0.0.1'.

• **Py4J Callback Server Socket Teardown:** PyDeequ launches a Py4J Gateway Callback Server for custom constraint lambdas. Under Windows Python, non-daemon gateway threads prevent processes from terminating upon script completion. We implemented explicit teardown hooks (cb.shutdown() and os._exit() in pytest lifecycle hooks) ensuring 100% clean terminal exit.

## 10. Automated Verification Suite & Production Recommendations

The repository includes an automated test suite (tests/test_pipeline.py) comprising 10 unit and integration tests. All 10 tests passed with 100% success rate: verifying deterministic seeding, exact fault injection rates, complete 4-category failure detection, quarantine inference blocking, and the subgroup B degradation inequality (acc_drop_B > acc_drop_A).

**Architectural Recommendations for Enterprise Feature Stores:**

1. **Never Rely on Type Systems Alone:** Unit and scale shifts produce valid numeric floats that pass schema validators; statistical range gates are essential.
2. **Disaggregate Model Health Metrics:** Continuously monitor fairness and accuracy across protected sub-populations rather than relying on global averages.
3. **Enforce Hard Quarantine Guards:** Integrate circuit-breaker checks into CI/CD and Airflow DAGs so corrupt batches are automatically isolated before inference.